



► Herramientas de Python para el álgebra lineal.

Bibliografía

- *Computational Physics*, Mark Newman.
- *Programming for computations - Python*, Evein Linge and Petter Langtangen.
- *Computational Physics with Python*, Eric Ayars.
- *Computational mathematics: An introduction to Numerical Analysis and Scientific Computing with Python*, Dimitrios Mitsotakis.
- *Computational physics: problem solving with Python*, Rubin H. Landau, Manuel J. Páez and Cristian C. Bordeianu.

Sistema de ecuaciones lineales

Las librerías de Python nos permiten abordar una multitud de problemas en computación científica. Por ejemplo, un sistema de ecuaciones algebraicas lineales. En el papel resolvemos sistemas triviales,

$$3x - 2y = 1, \quad x + y = 8,$$

cuya solución es $x = 17/5$ y $y = 23/5$. Esto puede llevarse a un sistema matricial

$$\begin{bmatrix} 3 & -2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 8 \end{bmatrix}$$

o de forma compacta como $Ax = v$. Encontrar las soluciones se puede hacer por reducción de Gauss, o por la inversa, o por la factorización LU. En caso de que exista la inversa de A , se tiene que $x = A^{-1}v$. La rutina `np.linalg.solve` de numpy nos ayuda con ello.

```
1 In [1]: import numpy as np
2
3 In [2]: A = np.array([[3, -2], [1, 1]])
4
5 In [3]: v = np.array([1, 8])
6
7 In [4]: np.linalg.solve(A, v)
8 Out[4]: array([3.4, 4.6])
9
10 In [5]: 17 / 5
```

```

11 Out [5]: 3.4
12
13 In [6]: 23 / 5
14 Out [6]: 4.6

```

El problema anterior claramente no amerita el uso de una rutina computacional. Sin embargo, consideremos el siguiente sistema de ecuaciones

$$\begin{aligned}
 2w + x + 4y + z &= -4 \\
 3w + 4x - y - z &= 3 \\
 w - 4x + y + 5z &= 9 \\
 2w - 2x + y + 3z &= 7
 \end{aligned}$$

que en forma matricial se escribe como

$$\begin{bmatrix} 2 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{bmatrix} \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} = \begin{bmatrix} -4 \\ 3 \\ 9 \\ 7 \end{bmatrix}.$$

Este sistema de 4×4 ya resulta tedioso para resolver manualmente mediante la eliminación de Gauss. En Python, la solución numérica se obtiene directamente con el mismo procedimiento:

sistema_ecuaciones.py

```

1  """ Aquí se muestran algunas funciones básicas para trabajar con matrices"""
2
3  import numpy as np
4
5  # definición de la matrix
6  A = np.array([[2,1,4,1], [3,4,-1,-1], [1,-4,1,5], [2,-2,1,3]], dtype=float)
7
8  # definición del vector
9  v = np.array([4,-3,9,7], dtype=float)
10
11 # ==== propiedades de la matriz=====
12 detA = np.linalg.det(A) # determinante
13 rango = np.linalg.matrix_rank(A) #rango
14 transpuestaA = np.transpose(A) # transpuesta también pued usar A.T
15 trazaA = np.trace(A) #traza
16
17 print("=== PROPIEDADES DE LA MATRIZ A ===")
18 print(f"Rango: {rango}")
19 print(f"Traza: {trazaA}")
20 print(f"Determinante: {detA:.2f}")
21 print(f"Transpuesta:\n{transpuestaA}\n")
22
23 if not np.isclose(detA,0):
24     print(f"El determinante de A es \n det A={detA: .2f}, diferente de cero"
25     )

```

```
26     # método de la inversa
27     invA = np.linalg.inv(A)
28     x = invA @ v # note el uso del operador @ antes se usaba np.dot(invA,v)
29     residuo1 = (A @ x) - v
30
31     print(f"\nSu inversa es \n A-1 = {invA}")
32     print(f"\nLa solución del sistema usando la inversa es \n x = {x}")
33     print(f"\nCon residuo = {residuo1}")
34
35     # solución del sistema usando np.linalg.solve
36     y = np.linalg.solve(A,v)
37     residuo2 = (A @ y) - v
38     print(f"\nPor otro lado, podemos encontrar directamente la solución al
39     sistema utilizando np.linalg.solve\n x = {y}")
40     print(f"\nCon residuo = {residuo2}")
41
42 else:
43     print("La matriz A tiene determinante nulo, por lo tanto no es
44     invertible")
```

Puede consultar las rutinas utilizadas directamente desde la documentación de numpy [dando clic aquí](#). Note el uso del operador @ para realizar el producto matricial. En versiones anteriores de numpy se utilizaba `np.dot()`.

A continuación se muestra la diferencia entre la operación * (producto de Hadamard) y el producto matricial @. Note que el producto escalar · también se puede efectuar con @

```
1 In [38]: M = np.array([[1, 2], [3, 4]])
2
3 In [39]: N = np.array([[5, 6], [7, 8]])
4
5 In [40]: N*M
6 Out[40]:
7 array([[ 5, 12],
8        [21, 32]])
9
10 In [41]: M*N
11 Out[41]:
12 array([[ 5, 12],
13        [21, 32]])
14
15 In [42]: N @ M
16 Out[42]:
17 array([[23, 34],
18        [31, 46]])
19
20 In [43]: M @ N
21 Out[43]:
22 array([[19, 22],
23        [43, 50]])
24
```

```

25 In [44]: r = np.array([1,8,9,10,12,-20],dtype=float)
26
27 In [45]: v = np.array([-1,8,-9,110,12,-20],dtype=float)
28
29 In [46]: print(r @ v)
30 1626.0
31
32 In [47]: print(v @ r)
33 1626.0

```

En álgebra lineal también se estudia la factorización LU. Suponga que quiere resolver $Ax = v$. Escribamos $A = LU$, donde L es triangular inferior y U es triangular superior. Luego, $Ax = LUx = v$ y se define $y = UX$, tal que $Ly = v$. Para la factorización LU, utilizamos la rutina `scipy.linalg.lu`

LUfactor.py

```

1  """ Aquí se muestran algunas funciones básicas para trabajar con matrices"""
2
3  import numpy as np
4  import scipy as sp
5
6  # definición de la matrix
7  A = np.array([[2,1,4,1], [3,4,-1,-1], [1,-4,1,5], [2,-2,1,3]], dtype=float)
8
9  # definición del vector
10 v = np.array([4,-3,9,7], dtype=float)
11
12 # ==== Descomposición LU====
13 # P es la matriz de permutación, L es triangular inferior y U triangular
   superior
14 P, L, U = sp.linalg.lu(A)
15
16 print("=== DESCOMPOSICIÓN LU (P A = L U) ===")
17 print(f"Matriz P (Permutación):\n{P}\n")
18 print(f"Matriz L (Triangular Inferior con 1s en la diagonal):\n{L}\n")
19 print(f"Matriz U (Triangular Superior):\n{U}\n")
20
21 # Verificación de la descomposición P @ L @ U == A
22 print(f"Verificación P @ L @ U == A:")
23 print(f"{P @ L @ U}\n")
24
25 # Resolución del sistema Ax = v usando LU (L y = P^T v, luego U x = y)
26 y_lu = sp.linalg.solve_triangular(L, P.T @ v, lower=True)
27 x_lu = sp.linalg.solve_triangular(U, y_lu, lower=False)
28
29 residuo1 = (A @ x_lu) -v
30 print(f"Solución x usando Descomposición LU:\nx = {x_lu}\n")
31 print(f"con residuo:\n {residuo1}\n")
32
33 if not np.isclose(np.linalg.det(A),0):
34     # solución del sistema usando np.linalg.solve

```

```

35     y = np.linalg.solve(A,v)
36     residuo2 = (A @ y) - v
37     print(f"Por otro lado, podemos encontrar directamente la solución al
38     sistema utilizando np.linalg.solve\n x = {y}")
39     print(f"Con residuo = {residuo2}")
40 else:
41     print("La matriz A tiene determinante nulo, por lo tanto no es
    invertible")

```

Puede consultar las rutinas desde la documentación de [scipy dando clic aquí](#).

Diagonalización de matrices

Finalmente, un problema fundamental que aparece en Física es el problema de autovalores y autovectores, $Av = \lambda v$. Prácticamente, la diagonalización de matrices es el pan de cada día en electromagnetismo, mecánica cuántica, mecánica clásica, entre otras.

autovalores_autovectores.py

```

1  """ Aquí se muestran las funciones básicas para calcular autovectores y
2  autovalores """
3  import numpy as np
4
5  # definición de la matrix
6  M = np.array([[2,1,4,1,7], [3,4,-1,1,0], [1,-4,1,5,2], [2,-2,1,3,6],
7  [-1,-3,2,1,-10] ], dtype=float)
8
9  # ==== Autovalores y autovectores propiedades de la matriz ====
10 autovalores, autovectores = np.linalg.eig(M)
11
12 print("==== AUTOVALORES Y AUTOVECTORES ===")
13 print(f"Autovalores ():\n{autovalores}\n")
14 print(f"Autovectores (columnas v_i):\n{autovectores}\n")
15
16 print("-----\n")
17 print("Verificación de la relación A * v = * v sin usar enumerate")
18 print("-----\n")
19
20 for i in range(len(autovalores)):
21     lamvi = autovalores[i]
22     vi = autovectores[:,i]
23     lamv = lamvi * vi
24     Mv = M @ vi
25     print(f"Autovalor _{i} = {lamvi:.4f}")
26     print(f" M @ v_{i}: {Mv}")
27     print(f" _{i} * v_{i}: {lamv}")
28     # Comprobación de igualdad numérica
29     print(f" ¿Son iguales? {np.allclose(Mv, lamv)}\n")

```

```

29
30 print("-----\n")
31 print("Verificación de la relación A * v =  * v usando enumerate")
32 print("-----\n")
33
34 for i, lam in enumerate(autovalores):
35     vi = autovectores[:, i]
36     Mv = M @ vi
37     lamv = lam * vi
38     print(f"Autovalor _{i} = {lam:.4f}")
39     print(f"  M @ v_{i}:      {Mv}")
40     print(f"  _{i} * v_{i}:    {lamv}")
41     # Comprobación de igualdad numérica
42     print(f" ¿Son iguales? {np.allclose(Mv, lamv)}\n")

```

Acá hemos hecho uso del comando `enumerate` para extraer el número asociado al autovalor. Por ejemplo,

```

1 In [76]: lista= ["Armenia", "Pereira", "Barbosa", "Girardota"]
2
3 In [77]: for i in lista:
4     ...:     print(i)
5     ...:
6 Armenia
7 Pereira
8 Barbosa
9 Girardota
10
11 In [78]: for i in enumerate(lista):
12     ...:     print(i)
13     ...:
14 (0, 'Armenia')
15 (1, 'Pereira')
16 (2, 'Barbosa')
17 (3, 'Girardota')
18
19 In [79]: calificaciones = [0.5, 3.2, 1.7, 4.8]
20
21 In [80]: for i, notas in enumerate(calificaciones):
22     ...:     print(f"índice: {i}, valor asiciado a ese índice: {notas}")
23     ...:
24 índice: 0, valor asiciado a ese índice: 0.5
25 índice: 1, valor asiciado a ese índice: 3.2
26 índice: 2, valor asiciado a ese índice: 1.7
27 índice: 3, valor asiciado a ese índice: 4.8

```

Para concluir esta parte, veamos la descomposición en valores singulares (*Singular Value Decomposition*, SVD). Este teorema nos permite diagonalizar cualquier matriz general de orden $m \times n$ mediante lo que en Física de Partículas denominamos una **transformación biunitaria**:

$$M = V_L \Sigma V_R^\dagger \implies \Sigma = V_L^\dagger M V_R,$$

donde V_L y V_R son matrices unitarias que rotan los campos de izquierda y derecha, respectivamente, y Σ contiene las masas físicas (valores singulares $\sigma_i \geq 0$).

Tabla 1: Paralelo entre la notación de física de partículas y la notación estándar de SVD en álgebra lineal.

Concepto	Física de Partículas	Matemática (SVD)	Interpretación
Matriz de masa / operador	$M \in \mathbb{C}^{m \times n}$	$A \in \mathbb{C}^{m \times n}$	Matriz de acoplamiento de sabor.
Masas / Valores singulares	$M_{\text{diag}} = \text{diag}(m_i)$	$\Sigma = \text{diag}(\sigma_i)$	Masas físicas ($\sigma_i = m_i \geq 0$).
Rotación izquierda (L)	$V_L \in U(m)$	$U \in U(m)$	Rotación de campos izquierdos.
Rotación derecha (R)	$V_R \in U(n)$	$V \in U(n)$	Rotación de campos derechos.
Transformación	$M_{\text{diag}} = V_L^\dagger M V_R$	$\Sigma = U^\dagger A V$	Diagonalización biunitaria.
Descomposición	$M = V_L M_{\text{diag}} V_R^\dagger$	$A = U \Sigma V^\dagger$	Reconstrucción SVD.
Matriz de mezcla	$U_{\text{PMNS}} = V_L^{e\dagger} V_L^\nu$	$W = U_1^\dagger U_2$	Matriz física de oscilación.

Considere, por ejemplo, la aplicación directa en Python tanto para una matriz de masa cuadrada 3×3 como para una matriz no cuadrada 3×4 :

Tbiunitaria.py

```

1 """ Aplicamos la transformación biunitaria para diagonalizar una matrix
2 en matemáticas se conoce como el singular value decomposition (SVD) """
3 import numpy as np
4
5
6 # Matriz de masa cuadrada (3 x 3)
7 M = np.array([[2.0 + 1j, 0.5 - 0.2j, 1.0], [0.1, 1.5 + 0.8j, 0.3j], [0.0,
8 0.2, 0.8 - 0.5j] ])
9
10 # Aplicamos SVD (Transformación biunitaria)
11 # U y V son unitarias (en partículas son V_L y V_R) Md es S, la diagonal
12 VL, Md, VRh = np.linalg.svd(M)
13
14 # contruimos el VR porque la rutina devuelve el hermítico
15 VR = VRh.conj().T
16
17 # Verificamos la diagonalización V_L^\dagger @ M @ V_R
18 M_diag = VL.conj().T @ M @ VR
19
20 print("==== matrix M cuadrada 3x3 =====")
21 print("Masas físicas (Valores singulares S con np.linalg.svd(M)):")
22 print(np.round(Md, 4))
23
24 print("\nMatriz diagonalizada V_L^H @ M @ V_R:")
25 print(np.round(M_diag, 4))
26
27 # Matriz de masa no cuadrada cuadrada (3 x 4)
28 N = np.array([[2.0 + 1j, 0.5 - 0.2j, 0.0, 1. -1j], [0.1, 1.5 + 0.8j, 0.3j,
29 1. -2j ], [0.0, 0.2, 0.8 - 0.5j, 1. +3j] ])
30 # Aplicamos SVD (Transformación biunitaria)

```

```

29 # U y V son unitarias (en partículas son V_L y V_R) Md es S, la diagonal
30 VNL, MNd, VNRh = np.linalg.svd(N)
31
32 # contruimos el VR porque la rutina devuelve el hermítico
33 VNR = VNRh.conj().T
34
35 # Verificamos la diagonalización V_L^\dagger @ M @ V_R
36 MN_diag = VNL.conj().T @ N @ VNR
37
38 print("\n===== matrix N no cuadrada 3x4 =====")
39 print(f"Dimensiones de N: {N.shape}")
40 print(f"Dimensiones de VNL: {VNL.shape} | VNR: {VNR.shape}")
41
42 print("Masas físicas (Valores singulares S con np.linalg.svd(MN):")
43 print(np.round(MNd, 4))
44
45 print("\nMatriz diagonalizada VN_L^H @ N @ VN_R:")
46 print(np.round(MN_diag, 4))
47 print(f"El rango de N es: {np.linalg.matrix_rank(N)} que equivale a las
    masas físicas")

```

Resumen de las rutinas básicas utilizadas

◆ Acceda a la documentación oficial scipy [dando clic aquí](#).

◆ Acceda a la documentación oficial scipy [dando clic aquí](#).

<code>detA = np.linalg.det(A)</code>	(Determinante)
<code>rango = np.linalg.matrix_rank(A)</code>	(Rango de la matriz)
<code>transpA = np.transpose(A) # o A.T</code>	(Matriz Transpuesta)
<code>trazaA = np.trace(A)</code>	(Traza de la matriz)
<code>invA = np.linalg.inv(A)</code>	(Inversa explícita)
<code>y = np.linalg.solve(A, v)</code>	(Solución directa de $Ay = v$)
<code>P, L, U = scipy.linalg.lu(A)</code>	(Descomposición LU con pivoteo)
<code>autovalores, autovectores = np.linalg.eig(M)</code>	(Diagonalización)
<code>VL, S, VRh = np.linalg.svd(M)</code>	(Valores Singulares)

1. `np.linalg.det(A)`

- **Qué hace:** Calcula el **determinante** de una matriz cuadrada.
- **Uso en física:** Indispensable para verificar si un sistema de ecuaciones tiene solución única ($\det(A) \neq 0$) o analizar volúmenes en transformaciones lineales.

2. `np.linalg.matrix_rank(A)`

- **Qué hace:** Determina el número de filas o columnas linealmente independientes empleando internamente descomposición SVD.

- **Interpretación:** Si el rango es menor que la dimensión de A , la matriz es singular (no invertible).

3. `np.transpose(A)` o `A.T`

- **Qué hace:** Intercambia filas por columnas ($A_{ij} \rightarrow A_{ji}$).
- **Nota:** Para matrices complejas en mecánica cuántica, se requiere la **traspuesta conjugada** (Daga $A^\dagger$), se debe usar `np.conj(A.T)` o `A.conj().T`.

4. `np.trace(A)`

- **Qué hace:** Suma los elementos de la diagonal principal ($\sum_i A_{ii}$).
- **Aplicación:** Conservación de invariantes bajo transformaciones unitarias en física.

1. `np.linalg.inv(A)` vs `np.linalg.solve(A, v)`

- **`np.linalg.inv(A)`:** Calcula la matriz inversa A^{-1} . Es numéricamente inestable y costoso computacionalmente si solo se quiere resolver un sistema $Ay = v$.
- **`np.linalg.solve(A, v)` (Recomendado):** Resuelve el sistema lineal $Ay = v$ directamente usando eliminación gaussiana sin calcular la inversa.
- **Regla de oro:** Siempre que necesites calcular $y = A^{-1}v$, utiliza `np.linalg.solve(A, v)` en lugar de multiplicar por la inversa.

2. `scipy.linalg.lu(A)`

- **Qué hace:** Descompone la matriz A en el producto $A = P \cdot L \cdot U$, donde:
 - **P:** Matriz de permutación (controla el pivoteo de filas).
 - **L:** Matriz triangular inferior con unos en la diagonal (*Lower*).
 - **U:** Matriz triangular superior (*Upper*).
- **Uso principal:** Resolver múltiples sistemas lineales que comparten la misma matriz de coeficientes A .

1. `np.linalg.eig(M)`

- **Qué devuelve:**
 - **autovalores:** Un arreglo 1D con los autovalores λ_i .
 - **autovectores:** Una matriz 2D donde las **columnas** representan los autovectores normalizados v_i .
- **Importante:** Satisface la ecuación $M \cdot v_i = \lambda_i v_i$. Si la matriz M es simétrica o hermitiana, es preferible utilizar `np.linalg.eigh(M)` por estabilidad numérica.

2. `np.linalg.svd(M)`

- **Qué es:** La Descomposición en Valores Singulares $M = V_L \cdot \Sigma \cdot V_R^\dagger$.
- **Estructura de salida:**
 - **VL (V_L):** Matriz unitaria izquierda.
 - **S (Σ):** Un arreglo 1D con los **valores singulares** (siempre reales y positivos $\sigma_i \geq 0$).
 - **VRh ($V_R^\dagger$):** La matriz transpuesta conjugada de la rotación derecha.
- **Uso en Física:** Diagonalización de matrices complejas no simétricas, masas de neutrinos (Majorana/Dirac) y reducción de dimensionalidad en datos.

Problemas:

1. Resuelva el siguiente sistema de ecuaciones lineales para las variables a hasta f :

$$\begin{aligned}\frac{a}{5} + b + c + d - e - f &= 24,1312 \\ a + c - d + 5e + f &= 46,2798 \\ -a - 3b + 2c - d - e - f &= -61,8372 \\ 5b - c + d + 2e &= 31,1466 \\ a - 2b + 3c + \frac{d}{2} &= 51,2106 \\ \frac{a}{2} - 2b - 2c - d + e - 6f &= -5,7008\end{aligned}$$

2. Considere la matriz:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}.$$

Utilizando los módulos NumPy y SciPy de Python:

- a) Calcule el determinante de la matriz A ($\det(A)$).
 - b) Determine la matriz inversa A^{-1} , en caso de que exista.
 - c) Haga la factorización LU.
 - d) Compute los autovalores y los correspondientes autovectores de A mediante la rutina `np.linalg.eig`.
3. En electrónica, las leyes de Kirchhoff se utilizan para resolver las corrientes que pasan a través de los componentes en redes de circuitos. Al aplicar estas leyes se obtienen sistemas de ecuaciones lineales, los cuales pueden expresarse como ecuaciones matriciales, tales como:

$$\begin{bmatrix} -13 & 2 & 4 \\ 2 & -11 & 6 \\ 4 & 6 & -15 \end{bmatrix} \begin{bmatrix} I_A \\ I_B \\ I_C \end{bmatrix} = \begin{bmatrix} 5 \\ -10 \\ 5 \end{bmatrix}$$

Usando el módulo de numpy, encuentre las corrientes I_A , I_B , I_C .

4. Los modos normales y las frecuencias angulares correspondientes para un sistema lineal de cuatro osciladores acoplados de masa m , conectados por resortes de igual constante elástica k , están dados por los autovectores y autovalores de la matriz M que se muestra a continuación:

$$M = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

Los autovalores representan el cuadrado o la escala de las frecuencias angulares ω en unidades de $\sqrt{k/m}$. Encuentre dichas frecuencias angulares.

Masas de neutrinos

Para ilustrar el papel del ordenamiento de autovalores y la diagonalización matricial en un contexto físico real, consideraremos el sector de neutrinos masivos de Majorana.

Sea M_ν una matriz de masa simétrica de 3×3 definida en la *base de sabor* $\{\nu_e, \nu_\mu, \nu_\tau\}$. En el lagrangiano de interacción, la masa se expresa como:

$$\mathcal{L}_{\text{mass}} = -\frac{1}{2} N_0^T M_\nu N_0 + \text{h.c.}, \quad \text{con } N_0 = \begin{pmatrix} \nu_e \\ \nu_\mu \\ \nu_\tau \end{pmatrix}.$$

Para pasar a los autoestados de masa $N = (\nu_1 \ \nu_2 \ \nu_3)^T$, realizamos la transformación unitaria de campos $N_0 = U N$, donde $U \equiv U_{\text{PMNS}}$ es la matriz de Pontecorvo-Maki-Nakagawa-Sakata:

$$\mathcal{L}_{\text{mass}} = -\frac{1}{2} (U N)^T M_\nu (U N) + \text{h.c.} = -\frac{1}{2} N^T (U^T M_\nu U) N + \text{h.c.}$$

Por consiguiente, la matriz de masa se diagonaliza mediante la transformación

$$M_{\text{diag}} = U^T M_\nu U = \begin{pmatrix} m_1 & 0 & 0 \\ 0 & m_2 & 0 \\ 0 & 0 & m_3 \end{pmatrix},$$

donde los autovalores $m_1, m_2, m_3 \geq 0$ representan las masas físicas de los neutrinos. En términos de componentes, la relación entre los estados de sabor $\alpha \in \{e, \mu, \tau\}$ y los estados de masa $i \in \{1, 2, 3\}$ se escribe:

$$\nu_\alpha = \sum_{i=1}^3 U_{\alpha i} \nu_i.$$

La matriz de mezcla U se parametriza habitualmente en términos de tres ángulos de rotación ($\theta_{12}, \theta_{23}, \theta_{13}$), una fase de violación CP (δ_{CP}) y dos fases de Majorana (α_{21}, α_{31}):

$$U = \begin{pmatrix} U_{e1} & U_{e2} & U_{e3} \\ U_{\mu 1} & U_{\mu 2} & U_{\mu 3} \\ U_{\tau 1} & U_{\tau 2} & U_{\tau 3} \end{pmatrix} = \begin{pmatrix} c_{12}c_{13} & s_{12}c_{13} & s_{13}e^{-i\delta_{\text{CP}}} \\ -s_{12}c_{23} - c_{12}s_{23}s_{13}e^{i\delta_{\text{CP}}} & c_{12}c_{23} - s_{12}s_{23}s_{13}e^{i\delta_{\text{CP}}} & s_{23}c_{13} \\ s_{12}s_{23} - c_{12}c_{23}s_{13}e^{i\delta_{\text{CP}}} & -c_{12}s_{23} - s_{12}c_{23}s_{13}e^{i\delta_{\text{CP}}} & c_{23}c_{13} \end{pmatrix} \text{diag} \left(1, e^{i\frac{\alpha_{21}}{2}}, e^{i\frac{\alpha_{31}}{2}} \right),$$

donde $c_{ij} \equiv \cos \theta_{ij}$ y $s_{ij} \equiv \sin \theta_{ij}$. A partir de los elementos de U , los ángulos de mezcla se aíslan directamente como:

$$\tan \theta_{12} = \left| \frac{U_{e2}}{U_{e1}} \right|, \quad \tan \theta_{23} = \left| \frac{U_{\mu 3}}{U_{\tau 3}} \right|, \quad \sin \theta_{13} = |U_{e3}|.$$

Restricciones experimentales y jerarquía de masas

Los experimentos de oscilación de neutrinos miden las diferencias de cuadrados de masa $\Delta m_{ij}^2 \equiv m_i^2 - m_j^2$ y los ángulos de mezcla. Las restricciones globales vigentes -[NuFIT 6.0](#)- imponen los

rangos a 3σ :

$$0,275 < \sin^2 \theta_{12} < 0,357, \quad 0,400 < \sin^2 \theta_{23} < 0,620, \quad 0,020 < \sin^2 \theta_{13} < 0,024,$$

$$6,82 \times 10^{-5} \text{ eV}^2 < \Delta m_{21}^2 < 8,03 \times 10^{-5} \text{ eV}^2.$$

Dado que los experimentos solares fijan la convención $m_1 < m_2$ (para que $\Delta m_{21}^2 > 0$), existen dos posibilidades físicamente distintas respecto al tercer autovalor m_3 :

1. **Jerarquía Normal (NO / NH):** $m_1 < m_2 < m_3$, donde $\Delta m_{31}^2 > 0$:

$$2,435 \times 10^{-3} \text{ eV}^2 < \Delta m_{31}^2 < 2,598 \times 10^{-3} \text{ eV}^2.$$

2. **Jerarquía Invertida (IO / IH):** $m_3 < m_1 < m_2$, donde $\Delta m_{32}^2 < 0$:

$$-2,581 \times 10^{-3} \text{ eV}^2 < \Delta m_{32}^2 < -2,414 \times 10^{-3} \text{ eV}^2.$$

Planteamiento del problema numérico: Independientemente de la física, el problema numérico es: Dada una matriz de masa $M_\nu \in \mathbb{C}_{3 \times 3}$ generada aleatoriamente:

- i) Obtener sus autovalores de masa y la matriz de rotación U mediante la descomposición $SV D$.
- ii) Ordenar los autovalores para identificar la jerarquía (NO o IO).
- iii) Verificar si los observables extraídos (Δm_{21}^2 , $|\Delta m_{3l}^2|$ y $\sin^2 \theta_{ij}$) se encuentran dentro de las cotas experimentales permitidas por NuFIT.